

Assignment 2

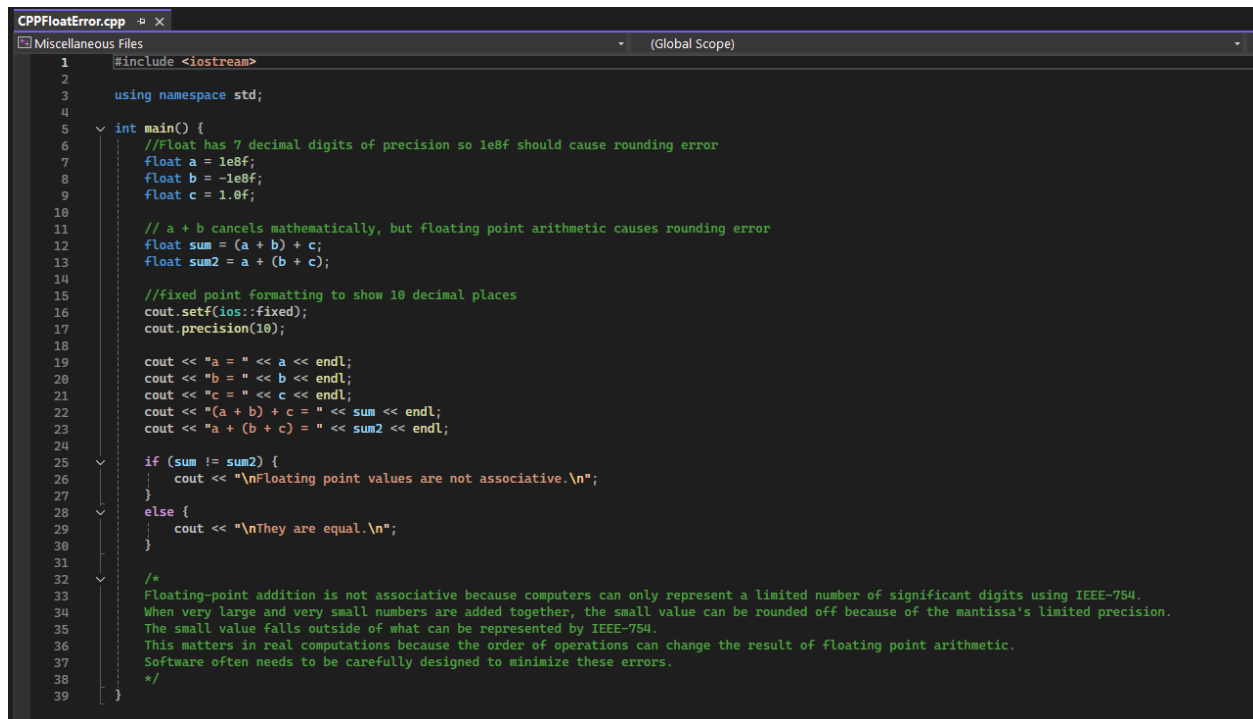
Grant Jones

9/21/2025

CS 570

Part 1:

CPP Code Floating-Point Addition:



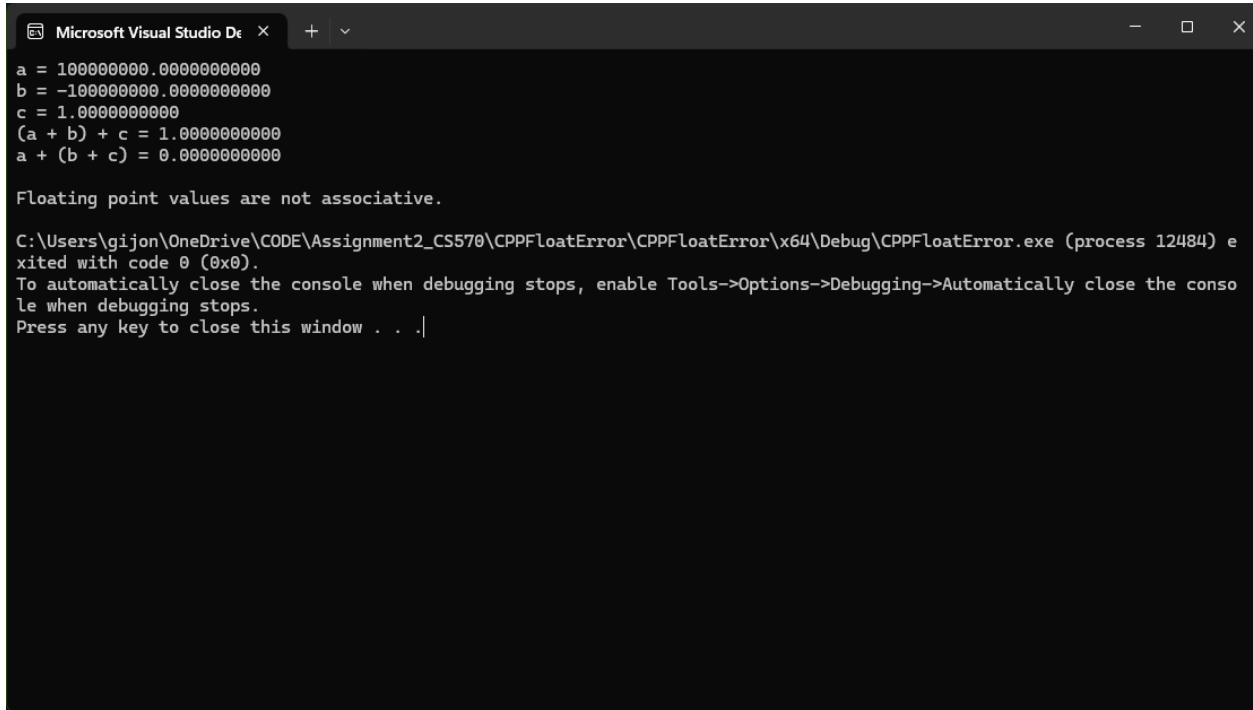
```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     //Float has 7 decimal digits of precision so 1e8f should cause rounding error
7     float a = 1e8f;
8     float b = -1e8f;
9     float c = 1.0f;
10
11     // a + b cancels mathematically, but floating point arithmetic causes rounding error
12     float sum = (a + b) + c;
13     float sum2 = a + (b + c);
14
15     //fixed point formatting to show 10 decimal places
16     cout.setf(ios::fixed);
17     cout.precision(10);
18
19     cout << "a = " << a << endl;
20     cout << "b = " << b << endl;
21     cout << "c = " << c << endl;
22     cout << "(a + b) + c = " << sum << endl;
23     cout << "a + (b + c) = " << sum2 << endl;
24
25     if (sum != sum2) {
26         cout << "\nFloating point values are not associative.\n";
27     }
28     else {
29         cout << "\nThey are equal.\n";
30     }
31
32     /*
33     Floating-point addition is not associative because computers can only represent a limited number of significant digits using IEEE-754.
34     When very large and very small numbers are added together, the small value can be rounded off because of the mantissa's limited precision.
35     The small value falls outside of what can be represented by IEEE-754.
36     This matters in real computations because the order of operations can change the result of floating point arithmetic.
37     Software often needs to be carefully designed to minimize these errors.
38     */
39 }
```

Floating-point addition is not associative because computers can only represent a limited number of significant digits using IEEE-754. When very large and very small numbers are added together, the small value can be rounded off because of the mantissa's limited precision. The small value falls outside of what can be represented by IEEE-754. This matters in real

computations because the order of operations can change the result of floating point arithmetic.

Software often needs to be carefully designed to minimize these errors.

Output:

A screenshot of a Visual Studio Debug Console window. The window has a title bar that says "Microsoft Visual Studio De" and standard window controls. The console output is as follows:

```
a = 100000000.0000000000
b = -100000000.0000000000
c = 1.0000000000
(a + b) + c = 1.0000000000
a + (b + c) = 0.0000000000

Floating point values are not associative.

C:\Users\gijon\OneDrive\CODE\Assignment2_CS570\CPPFloatError\CPPFloatError\x64\Debug\CPPFloatError.exe (process 12484) e
xited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .|
```

In my program, the reason $(a + b) + c$ and $a + (b + c)$ produce different results is due to how the operations are grouped and how floats store numbers. A float only has about seven decimal digits of precision. In the first case, $(a + b) + c$, the values $a = 1e8f$ and $b = -1e8f$ are added first. These cancel out to zero, and then when I add $c = 1.0f$, the result is 1.0 . In the second case, $a + (b + c)$, the addition starts with $b + c$. Since $b = -1e8f$, adding $1.0f$ does not change its value because $1.0f$ is too small compared to $100,000,000$ for a float to represent both. The result is effectively $-1e8f$. When a is then added, the large positive and large negative cancel, producing 0.0 . So in $(a + b) + c$ the small term c survives, but in $a + (b + c)$ it gets lost due to rounding. That is why my program prints different results and demonstrates that floating-point addition is not associative.

Part 2:

JavaScript:

[app.js](#):

```
app.js  ✎ ✕
Miscellaneous  <global>
1  const express = require('express');
2  const helmet = require('helmet');
3  const bodyParser = require('body-parser');
4  const path = require('path');
5
6  const app = express();
7  app.use(express.static('public'));
8  app.use(helmet());
9
10 app.set('view engine', 'ejs');
11 app.set('views', path.join(__dirname, 'views'));
12 app.use(bodyParser.urlencoded({ extended: false }));
13
14 const store = [];
15
16 app.get('/', (req, res) => {
17   const reflected = req.query.q || '';
18   res.render('index', { reflected, store });
19 });
20
21 app.post('/save', (req, res) => {
22   const message = req.body.message || '';
23   store.push({ message, time: new Date().toISOString() });
24   console.log('[LOG] Saved message: ${message}');
25   res.redirect('/');
26 });
27
28 const port = 3000;
29 app.listen(port, () => console.log(`Server running at http://localhost:3000`));
30
```

Index.ejs:

```
index.ejs  ✖
1  <!doctype html>
2  <html lang="en">
3  <head>
4    <meta charset="utf-8">
5    <title>XSS Safe Demo</title>
6  </head>
7  <body>
8    <h1>XSS Test: Assignment 2</h1>
9
10   <h2>Reflected XSS Example:</h2>
11   <p>Try URL: <code>?q=&lt;TEST&gt;</code></p>
12   <div id="reflected"><%= reflected %></div>
13
14   <h2>Stored Messages Example:</h2>
15   <form method="POST" action="/save">
16     <input name="message" placeholder="Enter message">
17     <button type="submit">Save</button>
18   </form>
19   <ul>
20     <%= store.forEach(item => { %>
21     <li><strong><%= item.time %>:</strong> <%= item.message %></li>
22     <%= }) %>
23   </ul>
24
25   <h2>DOM-based XSS Example:</h2>
26   <div id="domOutput"></div>
27   <script src="/js/dom.js"></script>
28
29   <script>
30     // DOM-based XSS safe insertion
31     window.addEventListener('DOMContentLoaded', () => {
32       const params = new URLSearchParams(window.location.search);
33       const user = params.get('user') || '';
34       const output = document.getElementById('domOutput');
35       if (output) {
36         output.textContent = user; // Safe: displays as text
37         console.log('[DOM] User param:', user); // Logs to console for debugging
38       } else {
39         console.error('DOM output element not found');
40       }
41     });
42   </script>
43 </body>
44 </html>
45
46
```

[dom.js:](#)

```
dom.js  ✖
Miscellaneous  <global>
1  window.addEventListener('DOMContentLoaded', () => {
2    const params = new URLSearchParams(window.location.search);
3    const user = params.get('user') || '';
4    const output = document.getElementById('domOutput');
5    if (output) {
6      output.textContent = user; // Safe
7      console.log('[DOM] User param:', user);
8    }
9  });
10
```

Discussion:

REFLECTED XSS:

Reflected XSS occurs when user input from the URL is immediately displayed on a webpage without proper checks. The route in app.js retrieves the q query and passes it to the index.ejs file:

JavaScript

```
app.get('/', (req, res) => {  
  
  const reflected = req.query.q || "";  
  
  res.render('index', { reflected, store });  
  
});
```

HTML

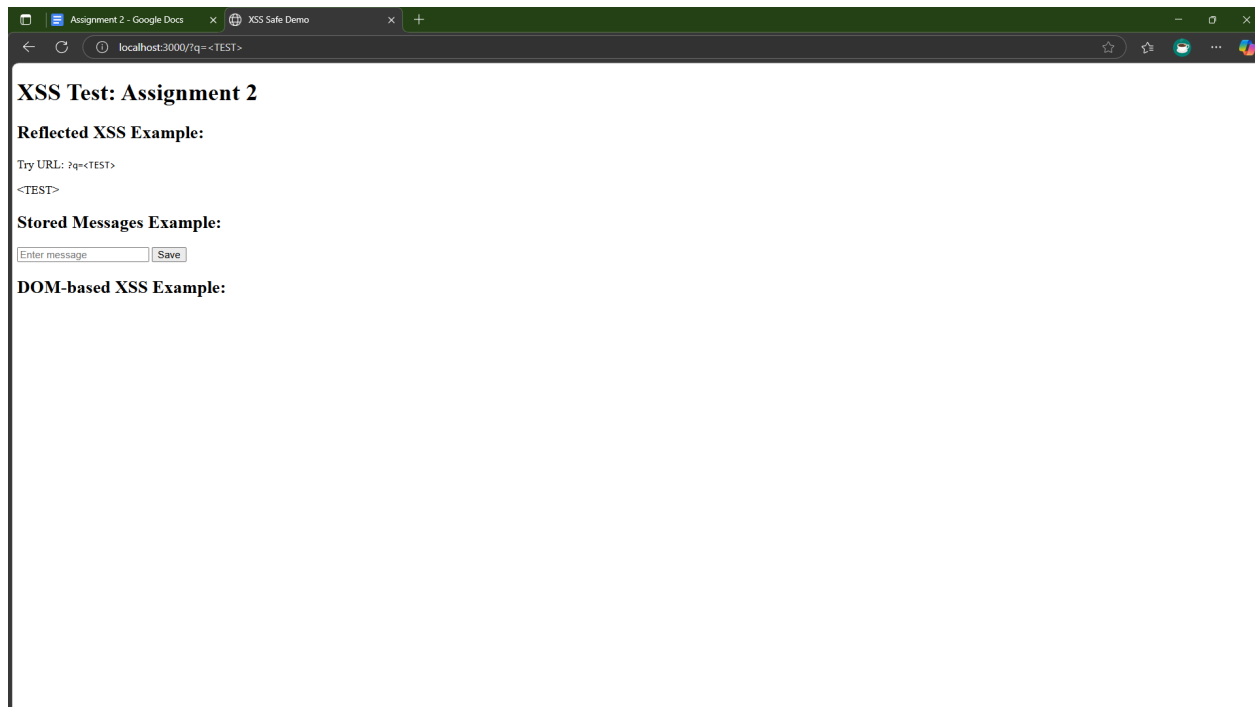
```
<div id="reflected"><%= reflected %></div>
```

In the HTML file, `<%= reflected %>` is used to automatically escape all HTML characters. It converts: `<` to `<`;

`>` to `>`;

`&` to `&`;

`"` to `"`;



So a URL input that would cause a breach such as `<script>alert('XSS')</script>` becomes `<script>alert('XSS')</script>`. The browser then displays this as text instead of running it. You can see in the Reflected_Protection screenshot how the site executed `localhost:3000/?<TEST>` as text rather than script under the Reflected XSS Example.

STORED XSS:

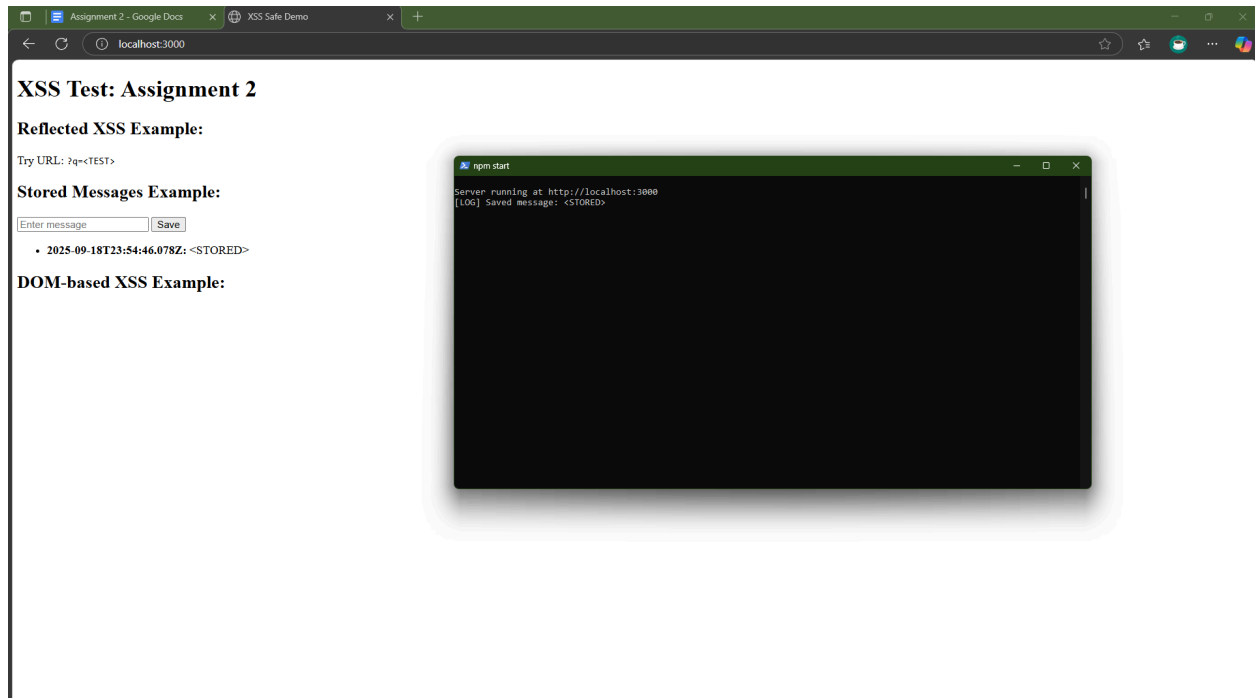
Stored XSS occurs when user-submitted data is saved on the server and later displayed to other users without proper checks. Users can submit messages through a form under the Stored XSS header that posts to the `/save` route in [app.js](#):

JavaScript:

```
app.post('/save', (req, res) => {  
  const message = req.body.message || "";  
  store.push({ message, time: new Date().toISOString() });  
  console.log(`[LOG] Saved message: ${message}`);  
  res.redirect('/');  
});
```

HTML:

```
<ul>  
  <% store.forEach(item => { %>  
    <li><strong><%= item.time %>:</strong> <%= item.message %></li>  
  <% }) %>  
</ul>
```



This section also checks for HTML characters in `<%= item.time %>` and `<%= item.message %>` where `<%= %>` escapes any HTML characters submitted in the message box. The Server logs the message through `console.log`. You can see in the screenshot Stored_Protection that the console window displays the message `<STORED>` as a way to log input.

DOM-based XSS:

DOM-based XSS occurs entirely in the browser when JavaScript inserts user input into the DOM. The `dom.js` script reads a user parameter from the URL and inserts it into a `<div>`:

JavaScript:

```
window.addEventListener('DOMContentLoaded', () => {  
  const params = new URLSearchParams(window.location.search);
```

```
const user = params.get('user') || "";

const output = document.getElementById('domOutput');

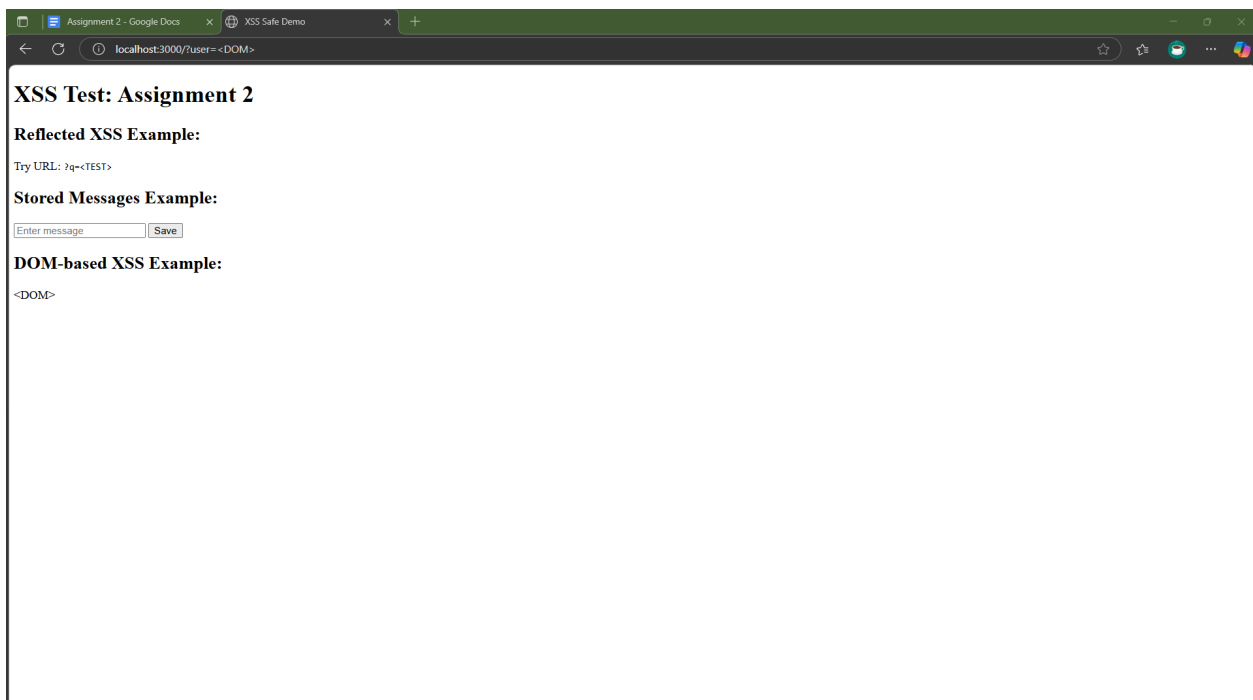
if (output) {

    output.textContent = user; // SAFE: displays as plain text

    console.log('[DOM] User param:', user);

}

});
```



Dom-based XSS happens entirely in the browser. This JavaScript protects against DOM-based XSS by using `output.textContent` rather than `output.innerHTML`. `innerHTML` parses and executes JavaScript and HTML directly, which would lead to XSS if the script was uploaded to the browser. `output.textContent` treats everything as plain text so `<script>` tags are displayed literally and not executed. The `console.log` also displays the exact value read from the

URL. You can see in the screenshot DOM_Protection that the <DOM> appeared as plain text rather than executing as script.

Part 3:

I attempted to write a more in depth example of buffer overflow in a separate cpp file, but Visual Studio now protects against buffer overflow and will not run the code if memory has been corrupted, even if using a strcpy. If you have a char array that holds a string such as:

```
char name[10];
```

Then ask for input from user:

```
cin >> name;
```

Buffer overflow can overwrite memory past the array bounds if there is no length check.

“name” can hold up to 9 characters + null terminator. To prevent this from happening, robust input validation is needed:

```
Input_Validation.cpp
Miscellaneous Files (Global Scope)
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  bool isValidName(const string& name);
7
8  int main() {
9      /*
10       * I attempted to write a more in depth example of buffer overflow in a separate cpp file,
11       * but Visual Studio now protects against buffer overflow and will not run the code if memory has been corrupted,
12       * even if using a strcpy.
13       *
14       * If you have a char array that holds a string such as:
15       char name[10];
16       * Then ask for input from user:
17       cin >> name;
18       * Buffer overflow can overwrite memory past the array bounds if there is no length check.
19       * name can hold up to 9 characters + null terminator.
20       * To prevent this from happening, robust input validation is needed:
21       */
22
23       string name;
24       cout << "Enter a name that is less than 10 characters and only contains letters: ";
25       getline(cin, name);
26
27       if (name.length() > 10) {
28           cerr << "Error: Name cannot be greater than 10 characters";
29           return 1;
30       }
31       if (!isValidName(name)) {
32           cerr << "Error: Name can only contain letters";
33           return 1;
34       }
35
36       cout << name << " is a valid name";
37       return 0;
38   }
39
40   bool isValidName(const string& name) {
41       for (int i = 0; i < name.length(); i++) {
42           char c = name[i];
43           //only allow letters
44           if (!isalpha(c)) {
45               return false;
46           }
47       }
48       return true;
49   }
50
```

Example with the name Christopher, which is 11 characters:

```
Microsoft Visual Studio De x + -
Enter a name that is less than 10 characters and only contains letters: christopher
Error: Name cannot be greater than 10 characters
C:\Users\gijon\OneDrive\CODE\Assignment2_CSS70\Input-Validation\Input_Validation\x64\Debug\Input_Validation.exe (process
24152) exited with code 1 (0x1).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the conso
le when debugging stops.
Press any key to close this window . . .|
```